Program-1

AIM: Simulation of Process Scheduling (FCFS)

Algorithum:

1. Start the program.

2. Declare the number of processes (n) and burst time array (bt).

3. Initialize waiting time of first process as 0.

4. For each process `i` from 1 to n-1:

   Calculate waiting time:

   `wt[i] = wt[i-1] + bt[i-1]`

5. For each process `i` from 0 to n-1:

   Calculate turnaround time:

   `tat[i] = wt[i] + bt[i]`

6. Display the process number, burst time, waiting time, and turnaround time in tabular form.

7. Stop the program.

CODE:

```
#include<stdio.h>

int main() {

   int n = 3;                    // Number of processes

   int bt[3] = {5, 3, 8};            // Burst times

   int wt[3], tat[3], i;

   wt[0] = 0;                     // First process waits 0

   // Calculate Waiting Time

   for(i = 1; i < n; i++) {

      wt[i] = wt[i-1] + bt[i-1];

   }

  // Calculate Turnaround Time

   for(i = 0; i < n; i++) {

      tat[i] = wt[i] + bt[i];
```

```
    }
    // Print Result Table
    printf("Process\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i, bt[i], wt[i], tat[i]);
    }
    return 0;
}
```

OUTPUT:

Process	BT	WT	TAT
P0	5	0	5
P1	3	5	8
P2	8	8	16

RESULT:

Simulation of Process Scheduling (FCFS) Was successfully executed

Program-2

Aim: To demonstrate the working of basic **system calls** such as **open, write, read, and close** using file handling functions in C (simulation).

Algorithm:

1. Start the program.
2. Create (open) a file using `fopen()` → acts like **open() system call**.
3. Write text into the file using `fprintf()` → acts like **write() system call**.
4. Close the file using `fclose()` → acts like **close() system call**.
5. Reopen the file in read mode.
6. Read contents using `fgetc()` → acts like **read() system call**.
7. Display the output on screen.
8. End program.

Code:

```
#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    printf("System Call Simulation\n");

    fp = fopen("sample.txt", "w");      // open()
    fprintf(fp, "Hello System Call");   // write()
    fclose(fp);                         // close()

    fp = fopen("sample.txt", "r");      // open() again

    printf("Reading file:\n");
    while((ch = fgetc(fp)) != EOF)      // read()
        printf("%c", ch);

    fclose(fp);                         // close()
    return 0;
}
```

Output:

```
System Call Simulation
Reading file:
Hello System Call
```

RESULT:
Demonstrating the working of basic system calls Was successfully executed.

PROGRAM-3

Aim:

To simulate a **deadlock situation** where two processes wait indefinitely for each other's resources.

Algorithm:

1. Create two resources `R1` and `R2`.
2. Process P1 locks R1 first.
3. Process P2 locks R2 first.
4. P1 waits for R2 (held by P2).
5. P2 waits for R1 (held by P1).
6. Both processes wait forever → **Deadlock Occurs**.

Code :

```
#include <stdio.h>

int main() {
    int R1 = 1, R2 = 1;

    printf("P1 locks R1\n");
    R1 = 0;

    printf("P2 locks R2\n");
    R2 = 0;

    printf("P1 waits for R2...\n");
    printf("P2 waits for R1...\n");

    printf("\n*** Deadlock Occurred ***\n");

    return 0;
}
```

Output:

```
P1 locks R1
P2 locks R2
P1 waits for R2...
P2 waits for R1...

*** Deadlock Occurred ***
```

RESULT:

Simulation of a deadlock situation Was successfully executed.

PROGRAM-4

Aim:

To implement the **FIFO Page Replacement Algorithm** and count the number of page faults.

Algorithm:

1. Start the program.
2. Input a reference string of pages.
3. Maintain a fixed-size frame array.
4. For each page in reference string:
    - If page not in frame → **Page Fault**
        - Replace page using **FIFO rule**
        - Insert page at current pointer
        - Move pointer circularly
    - If page already present → No fault
5. Count page faults and display frames at each step.
6. End program.

Code:

```
#include <stdio.h>

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4};
    int n = 8, frames[3] = {-1, -1, -1};
    int fault = 0, ptr = 0;

    printf("FIFO Page Replacement\n");

    for(int i=0; i<n; i++){
        int hit = 0;
        for(int j=0; j<3; j++)
            if(frames[j] == pages[i])
                hit = 1;

        if(!hit){
            frames[ptr] = pages[i];
            ptr = (ptr + 1) % 3;
            fault++;
        }

        printf("Page %d -> [%d %d %d]\n", pages[i], frames[0], frames[1],
frames[2]);
    }

    printf("\nTotal Page Faults = %d\n", fault);
    return 0;
}
```

Output:

```
FIFO Page Replacement
Page 7 -> [7 -1 -1]
Page 0 -> [7 0 -1]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]

Total Page Faults = 6
```

RESULT:

Implementation of the FIFO Page Replacement Algorithm Was successfully executed

PROGRAM-5

Aim: Implementation of disk scheduling algorithm

Algorithm

1. Start the program.
2. Take an array of disk requests.
3. Set the initial head position.
4. For each request:
    - Calculate `movement = abs(request - current_head)`.
    - Add movement to total.
    - Update `current_head = request`.
5. Display total head movement.
6. End program.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int req[] = {82, 170, 43, 140, 24, 16, 190};
    int n = 7, head = 50, total = 0;

    printf("FCFS Disk Scheduling\n");

    for(int i=0; i<n; i++){
        total += abs(req[i] - head);
        printf("Move from %d → %d = %d\n", head, req[i], abs(req[i] -
head));
        head = req[i];
    }

    printf("\nTotal Head Movement = %d\n", total);
    return 0;
}
```

Output:

```
FCFS Disk Scheduling
Move from 50 → 82 = 32
Move from 82 → 170 = 88
Move from 170 → 43 = 127
Move from 43 → 140 = 97
Move from 140 → 24 = 116
Move from 24 → 16 = 8
Move from 16 → 190 = 174

Total Head Movement = 642
```

RESULT:

Implementation of disk scheduling algorithm Was successfully executed

PROGRAM-6

Aim:

To implement **File Access Methods** such as **Sequential Access** and **Direct Access** using C programming.

Algorithm:

A) Sequential Access

1. Open a file in write mode.
2. Write some sample data into the file.
3. Close and reopen the file in read mode.
4. Read characters one by one sequentially.
5. Display the content.

B) Direct (Random) Access

1. Open the same file in read mode.
2. Move file pointer to a specific position using `fseek()`.
3. Read the character at that position.
4. Display the character.

Code:

```
#include <stdio.h>

int main() {
    FILE *fp;
    char c;

    fp = fopen("data.txt", "w");
    fprintf(fp, "ABCDE12345");
    fclose(fp);

    printf("Sequential Access:\n");
    fp = fopen("data.txt", "r");
    while((c = fgetc(fp)) != EOF)
        printf("%c ", c);
    fclose(fp);

    printf("\n\nDirect Access:\n");
    fp = fopen("data.txt", "r");
    fseek(fp, 5, SEEK_SET); // Move to 6th character
    c = fgetc(fp);
    printf("Character at position 6 = %c\n", c);
    fclose(fp);

    return 0;
}
```

Output:

```
Sequential Access:
A B C D E 1 2 3 4 5

Direct Access:
Character at position 6 = 1
```

RESULT:

Implementation **File Access Methods** such as **Sequential Access** and **Direct Access** using C programming Was successfully executed.

PROGRAM-7

Aim:

To simulate **Distributed Scheduling** by distributing tasks across multiple nodes and selecting the node with the **least load** for each new task.

Algorithm:

1. Start the program and define an array representing **3 distributed nodes**.
2. Assign initial load values to each node.
3. For each incoming task:
    - Find the node with **minimum load**.
    - Assign the task to that node.
    - Increase its load.
4. After all tasks are assigned, display final loads of each node.
5. End the program.

Code :

```
#include <stdio.h>

int main() {
    int node[3] = {3, 5, 2};    // initial loads
    int tasks[5] = {1, 1, 1, 1, 1}; // 5 new tasks

    printf("Distributed Scheduling Simulation\n");

    for(int i=0; i<5; i++){
        int minNode = 0;
        for(int j=1; j<3; j++)
            if(node[j] < node[minNode])
                minNode = j;

        node[minNode] += tasks[i];
        printf("Task %d assigned to Node %d\n", i+1, minNode+1);
    }

    printf("\nFinal Loads:\nNode1=%d  Node2=%d  Node3=%d\n",
           node[0], node[1], node[2]);

    return 0;
}
```

Output:

```
Distributed Scheduling Simulation
Task 1 assigned to Node 3
Task 2 assigned to Node 1
Task 3 assigned to Node 1
Task 4 assigned to Node 3
Task 5 assigned to Node 3

Final Loads:
Node1 = 5  Node2 = 5  Node3 = 5
```

RESULT: Simulation of Distributed Scheduling by distributing tasks across multiple nodes Was successfully executed.

PROGRAM-8

Aim:

To simulate **Clock Synchronization** in a distributed system using the **Berkeley Algorithm**.

Algorithm:

1. Take clock times of **3 slave nodes**.
2. The master reads all clock values.
3. Compute the **average time**.
4. Calculate the **offset** for each node:
   - `offset = average - node_time`
5. Display new synchronized times after adjustment.

Code:

```
#include <stdio.h>

int main() {
    int clocks[3] = {10, 12, 14};  // slave clocks
    int sum = 0, avg;

    printf("Clock Synchronization (Berkeley Algorithm)\n");

    for(int i=0; i<3; i++)
        sum += clocks[i];

    avg = sum / 3;

    printf("Average Time = %d\n", avg);

    printf("\nAdjusted Clocks:\n");
    for(int i=0; i<3; i++)
        printf("Node %d -> New Time = %d\n", i+1, avg);

    return 0;
}
```

Output:

```
Clock Synchronization (Berkeley Algorithm)
Average Time = 12

Adjusted Clocks:
Node 1 -> New Time = 12
Node 2 -> New Time = 12
Node 3 -> New Time = 12
```

RESULT:
Simulate Clock Synchronization in a distributed system using the Berkeley Algorithm Was successfully executed.

PROGRAM-9

Aim:

To simulate **Concurrency Control** in a database using the **Two-Phase Locking (2PL) protocol**, ensuring serializability in concurrent transactions.

Algorithm:

1. Start two transactions `T1` and `T2`.
2. Each transaction requests locks on data items.
3. If a data item is free → grant lock.
4. If locked by another transaction → **transaction waits**.
5. After completing operations, transaction releases all locks.
6. Display execution order and lock states.

Code:

```
#include <stdio.h>

int main() {
    int A = 1, B = 1; // 1 = unlocked, 0 = locked

    printf("Two-Phase Locking (2PL) Simulation\n\n");

    printf("T1 requests lock on A\n");
    A = 0;
    printf("Lock granted to T1 on A\n");

    printf("T2 requests lock on B\n");
    B = 0;
    printf("Lock granted to T2 on B\n");

    printf("T1 requests lock on B (but B already locked by T2)\n");
    printf("T1 must wait...\n");

    printf("T2 releases lock on B\n");
    B = 1;

    printf("Now T1 gets lock on B\n");
    B = 0;

    printf("\nBoth transactions complete.\n");
    printf("*** Serial execution ensured by 2PL ***\n");

    return 0;
}
```

Output:

```
Two-Phase Locking (2PL) Simulation

T1 requests lock on A
Lock granted to T1 on A
T2 requests lock on B
Lock granted to T2 on B
```

```
T1 requests lock on B (but B already locked by T2)
T1 must wait...
T2 releases lock on B
Now T1 gets lock on B

Both transactions complete.
*** Serial execution ensured by 2PL ***
```

RESULT:

Simulation Concurrency Control in a database using the Two-Phase Locking (2PL) protocol Was successfully executed.

PROGRAM-10

AIM:

To write a program using POSIX Real-Time Threads that creates two threads that run concurrently.

Algorithum:

1. Start the program.
2. Include the pthread library.
3. Create two thread functions.
4. In `main()`, create two POSIX threads using `pthread_create()`.
5. Allow both threads to run concurrently.
6. Wait for both threads to finish using `pthread_join()`.
7. End the program.

Code:

```
#include <stdio.h>
#include <pthread.h>

void* task1(void* arg) {
    printf("Thread 1 is running...\n");
    return NULL;
}

void* task2(void* arg) {
    printf("Thread 2 is running...\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, task1, NULL);
    pthread_create(&t2, NULL, task2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Both threads finished.\n");
    return 0;
}
```

Output:

```
Thread 1 is running...
Thread 2 is running...
Both threads finished.
```

RESULT:

Program using POSIX Real-Time Threads Was successfully executed

PROGRAM-11

Aim:

To implement a **real-time simulation**

Algorithm:

1. Start the program and display “Simulation Started”.
2. Initialize the total simulation steps (e.g., 10).
3. Run a loop for each time unit:
    - Print current time.
    - Print progress percentage.
    - Wait for 1 second to simulate real-time.
4. End the loop and display “Simulation Completed”.

Code :

```
#include <stdio.h>
#include <windows.h>

int main() {
   printf("Real-Time Simulation Started\n");
   int total = 10;

   for(int t=1; t<=total; t++){
      printf("Time: %d seconds - Simulation running... ", t);
      printf("Progress: %d%%\n", (t*100)/total);
      Sleep(1000); // 1 second delay
   }

   printf("Simulation Completed.\n");
   return 0;
}
```

Output:

Real-Time Simulation Started

Time: 1 seconds - Simulation running... Progress: 10%

Time: 2 seconds - Simulation running... Progress: 20%

Time: 3 seconds - Simulation running... Progress: 30%

...

Time: 10 seconds - Simulation running... Progress: 100%

Simulation Completed.

RESULT:

Implementation of a real-time simulation Was successfully executed

PROGRAM-12

AIM:

To simulate a Hybrid Real-Time Scheduler that combines **Rate Monotonic Scheduling (RMS)** and **Earliest Deadline First (EDF)** for selecting the highest-priority task.

Algorithum:

1. Define tasks with execution time, period, and deadline.
2. Sort tasks using RMS (smaller period → higher priority).
3. If two tasks have the same period, apply EDF (earlier deadline → higher priority).
4. Select the task with highest hybrid priority.
5. Display the selected task.

Code:

```
#include <stdio.h>

struct Task { int id, exec, period, deadline; };

int main() {
    struct Task t[3] = {{1,2,5,7}, {2,1,5,6}, {3,1,8,10}};
    int hp = 0;

    for(int i=1;i<3;i++) {
        if(t[i].period < t[hp].period) hp = i;
        else if(t[i].period == t[hp].period && t[i].deadline <
t[hp].deadline)
            hp = i;
    }

    printf("Hybrid Scheduler selects Task %d\n", t[hp].id);
    return 0;
}
```

Output:

```
Hybrid Scheduler selects Task 2
```

RESULT:

simulate a Hybrid Real-Time Scheduler that combines Rate Monotonic Scheduling (RMS) and Earliest Deadline First (EDF)Was successfully executed

PROGRAM-13

Aim:

To simulate the power management approaches used in **Android (Doze & App Standby)** and **iOS (Low Power Mode)**, and observe how the system adapts task execution and battery usage.

Algorithm:

1. Initialize battery level and idle time counter.
2. For each time unit (timestep):
   - Check **idle time** and **battery level**.
   - If `idle > 10` → Android Doze → pause background tasks.
   - Else if `idle > 5` → Android App Standby → delay background tasks.
   - Else if `battery < 25` → iOS Low Power Mode → throttle CPU.
   - Else → Normal Mode → run all tasks.
3. Update battery consumption according to the mode.
4. Reset idle time periodically to simulate user activity.
5. Repeat until simulation ends.

Code :

```
#include <stdio.h>

int main() {
    int battery = 40, idle = 0;

    for(int t=0; t<20; t++) {
        printf("t=%d Battery=%d%% Idle=%d -> ", t, battery, idle);

        if(idle > 10) {
            printf("Android Doze: Background Blocked\n");
            battery -= 1;
        }
        else if(idle > 5) {
            printf("Android App Standby: BG Delayed\n");
            battery -= 2;
        }
        else if(battery < 25) {
            printf("iOS Low Power Mode: CPU Throttled\n");
            battery -= 2;
        }
        else {
            printf("Normal Mode: Full Performance\n");
            battery -= 3;
        }

        idle++;
        if(t % 6 == 0) idle = 0;    // simulate user activity
        battery -= 1;              // passive drain
    }
    return 0;
}
```

Output:

```
t=0 Battery=40% Idle=0 -> Normal Mode: Full Performance
t=1 Battery=36% Idle=1 -> Normal Mode: Full Performance
t=6  Battery=28% Idle=0 -> Normal Mode: Full Performance
t=12 Battery=18% Idle=6 -> iOS Low Power Mode: CPU Throttled
t=18 Battery=8%  Idle=6 -> iOS Low Power Mode: CPU Throttled
```

RESULT:

simulate the power management approaches used in Android (Doze & App Standby) and iOS (Low Power Mode)Was successfully executed

PROGRAM-14

Aim: To simulate the behavior of a simple Android application in C, demonstrating **UI interaction, button click, and event handling** in console environment.

Algorithm:

1. Initialize **TextView message** as a string.
2. Display a menu simulating the **UI**.
3. Show a **button option** to the user.
4. Take user input to simulate **button click**.
5. Update the **TextView message** based on user input.
6. Loop until user exits.

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char textView[50] = "Welcome!";
    int choice;

    while(1) {
        printf("\n--- Android App Simulation ---\n");
        printf("TextView: %s\n", textView);
        printf("1. Click Button\n");
        printf("2. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
```

```
        if(choice == 1) {
            strcpy(textView, "Button Clicked! Hello Android.");
        } else if(choice == 2) {
            printf("Exiting App Simulation.\n");
            break;
        } else {
            printf("Invalid choice. Try again.\n");
        }
    }
    return 0;
}
```

Output:

```
--- Android App Simulation ---
TextView: Welcome!
1. Click Button
2. Exit
Enter choice: 1

--- Android App Simulation ---
TextView: Button Clicked! Hello Android.
1. Click Button
2. Exit
Enter choice: 2
Exiting App Simulation.
```

RESULT:

simulate the behavior of a simple Android application in C, demonstrating UI interaction, button click, and event handling in console environment. Was successfully executed